

LAPORAN DATA WRANGLING

“MODUL SQLITE”

Disusun Untuk Memenuhi Tugas
Mata Kuliah Reguler Data Wrangling



Dosen Pengajar:
Kartika Maulida Hindrayani, S.Kom., M.Kom.

Disusun Oleh:
Nadia Siti Ramadhani / 25083010053
A083

PROGRAM STUDI SAINS DATA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN” JAWA TIMUR
2026

➤ Proses Instalasi SQLITE

Langkah 1: Membuka Website Resmi

Pertama, pengguna membuka browser (Google Chrome atau sejenisnya), kemudian mengakses situs resmi DB Browser for SQLite melalui link berikut:

<https://sqlitebrowser.org/dl/>

Halaman ini menampilkan berbagai pilihan download sesuai sistem operasi.

Downloads

(Please consider sponsoring us on Patreon 😊)

Windows

25083010053_Nadia Siti Ramadhani

Our latest release (3.13.1) for Windows:

- [DB Browser for SQLite – Standard installer for 32-bit Windows](#)
- [DB Browser for SQLite – .zip \(no installer\) for 32-bit Windows](#)
- [DB Browser for SQLite – Standard installer for 64-bit Windows](#)
- [DB Browser for SQLite – .zip \(no installer\) for 64-bit Windows](#)
- [DB Browser for SQLite – Standard installer for ARM64 Windows](#)
- [DB Browser for SQLite – .zip \(no installer\) for ARM64 Windows](#)

Free code signing provided by [SignPath.io](#), certificate by [SignPath Foundation](#).

Langkah 2: Memilih Versi yang Sesuai

Selanjutnya, pada bagian Windows, kita dapat memilih versi DB Browser for SQLite – Standard installer for 64-bit Windows. Pemilihan ini disesuaikan dengan sistem operasi yang digunakan, yaitu Windows 64-bit.

Downloads

(Please consider sponsoring us on Patreon 😊)

Windows

25083010053_Nadia Siti Ramadhani

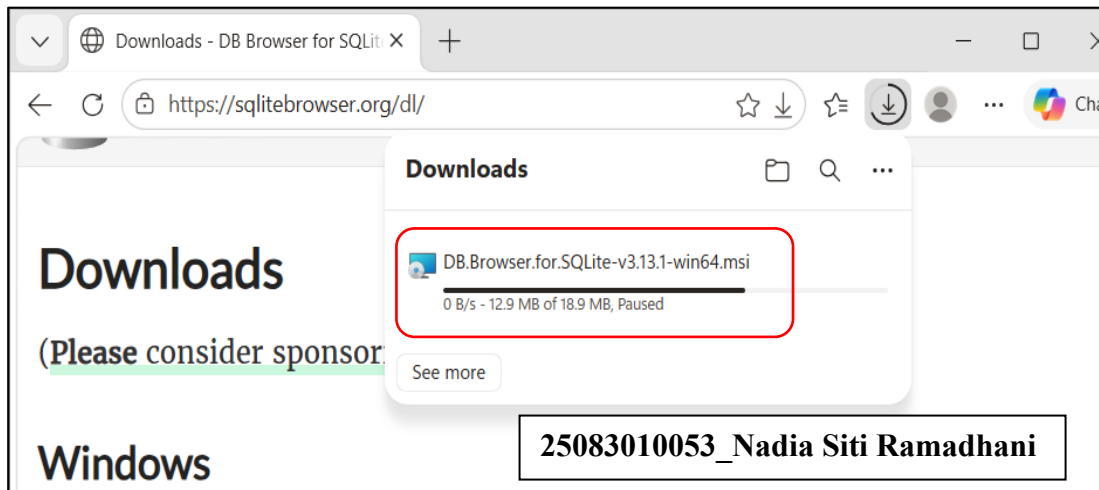
Our latest release (3.13.1) for Windows:

- [DB Browser for SQLite – Standard installer for 32-bit Windows](#)
- [DB Browser for SQLite – .zip \(no installer\) for 32-bit Windows](#)
- [DB Browser for SQLite – Standard installer for 64-bit Windows](#)
- [DB Browser for SQLite – .zip \(no installer\) for 64-bit Windows](#)
- [DB Browser for SQLite – Standard installer for ARM64 Windows](#)
- [DB Browser for SQLite – .zip \(no installer\) for ARM64 Windows](#)

Free code signing provided by [SignPath.io](#), certificate by [SignPath Foundation](#).

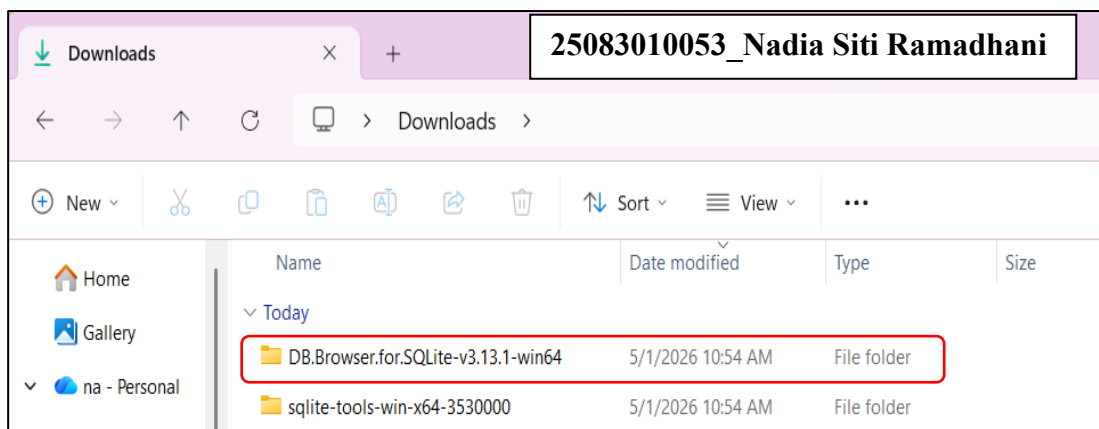
Langkah 3: Proses Download

Setelah mengklik pilihan tersebut, proses download dimulai secara otomatis. File installer dengan format .exe akan terlihat sedang diunduh pada browser.



Langkah 4: Hasil Download

Setelah proses download selesai, file akan tersimpan di folder Downloads pada komputer. File yang berhasil diunduh bernama DB.Browser.for.SQLite-3.13.1-win64

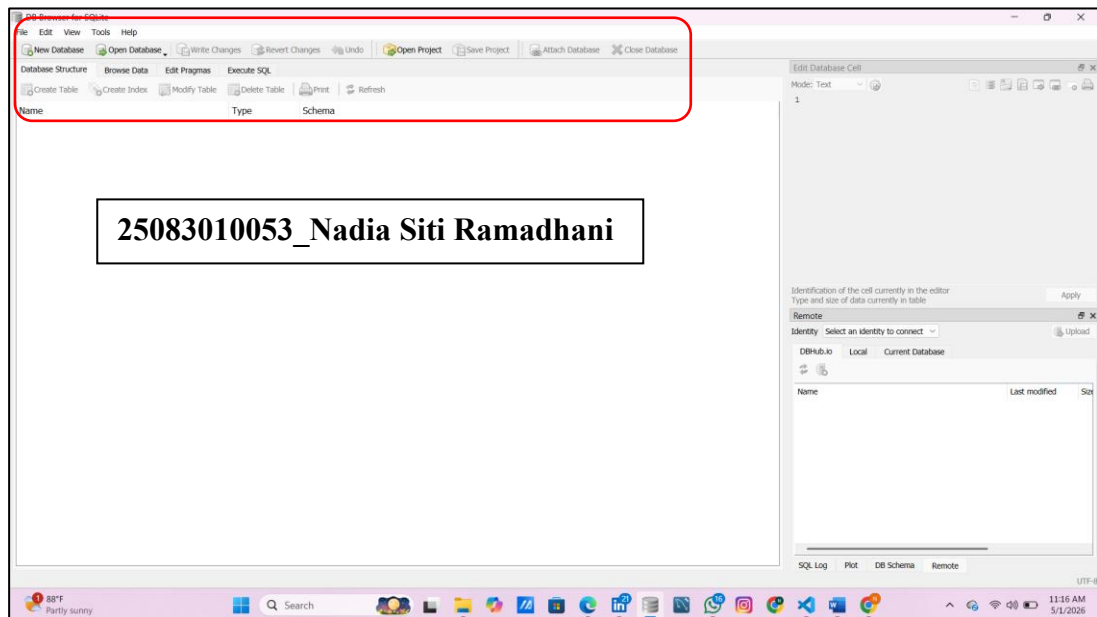


Langkah 5: Membuka Aplikasi SQLite

Setelah proses download selesai, pengguna menjalankan file installer dan menyelesaikan proses instalasi. Setelah itu, aplikasi DB Browser for SQLite dapat dibuka melalui menu Start atau shortcut di desktop.

Saat aplikasi berhasil dijalankan, akan muncul tampilan utama DB Browser for SQLite yang berisi beberapa menu seperti Database Structure, Browse Data, Edit Pragma, dan

Execute SQL. Tampilan ini digunakan untuk membuat, membuka, dan mengelola database SQLite.



Berdasarkan langkah-langkah yang telah dilakukan, proses pengunduhan dan instalasi DB Browser for SQLite berhasil dilakukan dengan baik. Pemilihan versi yang sesuai dengan sistem operasi sangat penting agar aplikasi dapat berjalan dengan optimal. Setelah aplikasi berhasil dibuka, pengguna dapat mulai menggunakan SQLite untuk membuat dan mengelola database.

➤ Koneksi Python dengan SQL

Langkah 1: Membuat Koneksi Database di Python

Pada gambar pertama, ditampilkan proses pembuatan koneksi antara Python dan SQLite menggunakan library sqlite3. Kode yang digunakan adalah:

```
import sqlite3
```

```
# membuat koneksi SQL ke database SQLite  
con = sqlite3.connect("pets.db")
```

```
# pastikan untuk menutup koneksi  
con.close()
```

25083010053_Nadia Siti Ramadhani

Penjelasan:

- import sqlite3 untuk mengakses database SQLite di Python
- sqlite3.connect('pets.db') membuat atau membuka database bernama pets.db

- `cursor = conn.cursor()` digunakan untuk menjalankan perintah SQL
- `conn.close()` menutup koneksi setelah selesai

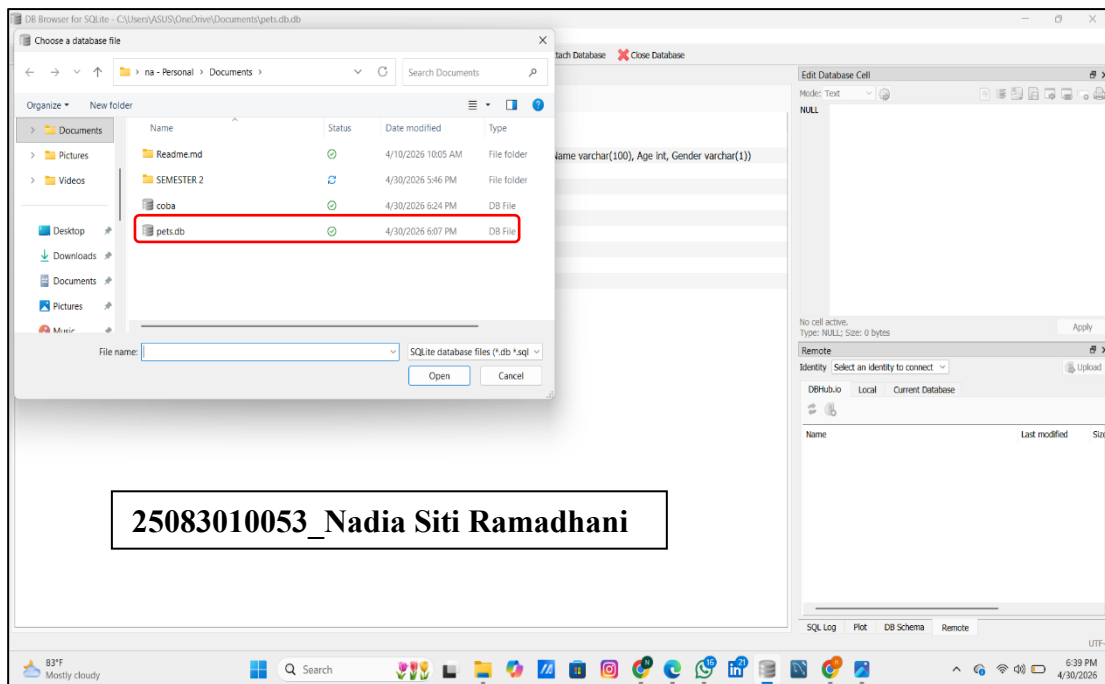
Langkah 2: Database Berhasil Dibuat

Pada gambar kedua, terlihat bahwa file database `pets.db` telah berhasil dibuat. File tersebut memiliki tipe DB File, yang menandakan bahwa database SQLite sudah berhasil dibuat melalui Python.

Name	Date modified	Type	Size
 <code>pets.db</code>	4/30/2026 5:18 PM	DB File	0 KB

Langkah 3: Membuka Database di DB Browser for SQLite

Pada gambar ketiga, ditampilkan proses membuka database menggunakan aplikasi DB Browser for SQLite. Kita dapat memilih file `pets.db` melalui menu *Open Database*, sehingga database dapat dilihat dan dikelola melalui tampilan GUI.



25083010053_Nadia Siti Ramadhani

Berdasarkan langkah-langkah yang telah dilakukan, Python berhasil terhubung dengan SQLite dan membuat database baru bernama `pets.db`. Database tersebut kemudian dapat dibuka menggunakan DB Browser for SQLite untuk pengelolaan lebih lanjut.

➤ Membuat Tabel Menggunakan Python

Langkah 1: Membuat Tabel Menggunakan Python

Pada tahap pertama, dilakukan pembuatan tabel database menggunakan bahasa pemrograman Python dengan library sqlite3. Program diawali dengan mengimpor library sqlite3, kemudian membuat koneksi ke database bernama **pets.db** menggunakan perintah:

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# digunakan untuk membuat tabel pet
for row in cur.execute("CREATE TABLE Pet (PetID int, PetName varchar(25), OwnerName varchar(100))"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

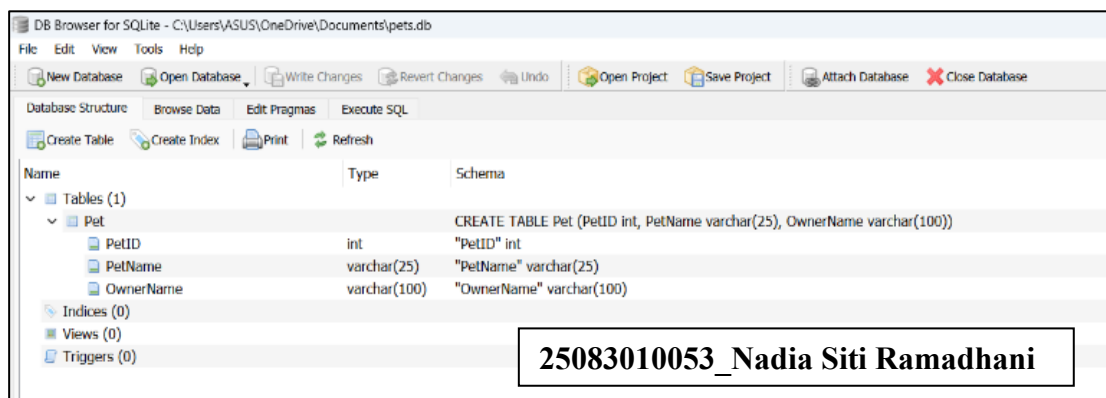
Setelah koneksi berhasil, dibuat objek cursor yang berfungsi untuk mengeksekusi perintah SQL. Selanjutnya dijalankan perintah CREATE TABLE untuk membuat tabel bernama Pet dengan struktur sebagai berikut:

- PetID bertipe integer
- PetName bertipe varchar(25)
- OwnerName bertipe varchar(100)

Perintah ini kemudian dijalankan menggunakan cursor.execute() dan perubahan disimpan dengan commit(), lalu koneksi database ditutup menggunakan close().

Langkah 2: Tampilan Hasil Tabel pada DB Browser for SQLite

Setelah program Python dijalankan, database **pets.db** dibuka menggunakan aplikasi **DB Browser for SQLite** untuk memastikan tabel berhasil dibuat.



25083010053_Nadia Siti Ramadhani

Pada tampilan **Database Structure**, terlihat bahwa tabel **Pet** telah berhasil dibuat dan memiliki tiga kolom, yaitu:

- PetID (int)
- PetName (varchar(25))
- OwnerName (varchar(100))

Hal ini menunjukkan bahwa proses pembuatan tabel melalui Python telah berhasil dilakukan.

➤ Menambahkkan Data Pada Tabel Dengan Python

Langkah 1: Menjalankan Perintah INSERT Menggunakan Python

Setelah tabel berhasil dibuat, tahap berikutnya adalah menambahkan data ke dalam tabel Pet menggunakan Python.

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk mengisi atau memasukkan nilai ke dalam tabel pet
for row in cur.execute("INSERT INTO Pet VALUES (1, 'Felix', 'Alex')"):
    print(row)
for row in cur.execute("INSERT INTO Pet VALUES (2, 'Auburn', 'Britt')"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

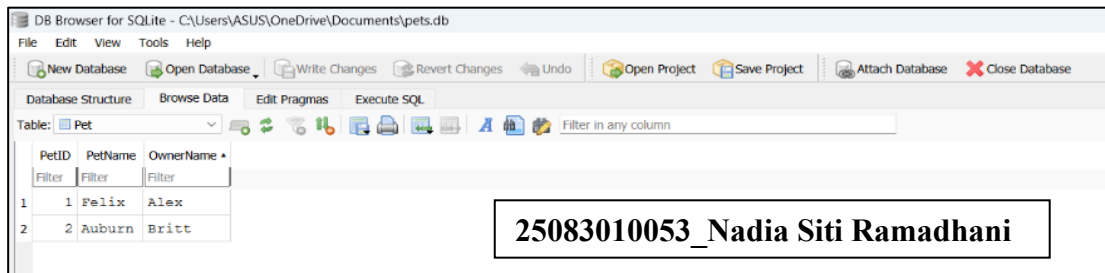
Program kembali mengimpor library sqlite3 dan melakukan koneksi ke database pets.db. Setelah koneksi dibuat, program menjalankan dua perintah SQL INSERT INTO untuk menambahkan data ke dalam tabel, yaitu:

- Data pertama:
PetID = 1, PetName = 'Felix', OwnerName = 'Alex'
- Data kedua:
PetID = 2, PetName = 'Auburn', OwnerName = 'Britt'

Perintah cursor.execute() digunakan untuk menjalankan query, kemudian perubahan disimpan dengan commit() dan koneksi database ditutup menggunakan close().

Langkah 2: Verifikasi Data pada DB Browser for SQLite

Setelah program dijalankan, database dibuka kembali menggunakan DB Browser for SQLite untuk memastikan data telah berhasil ditambahkan.



Pada menu Browse Data, terlihat bahwa tabel Pet telah berisi dua baris data sesuai dengan perintah INSERT yang telah dijalankan sebelumnya.

➤ Menampilkan Data dari Database SQLite

Menjalankan Query SELECT Menggunakan Python

Tahap selanjutnya adalah menampilkan seluruh data yang tersimpan pada tabel Pet.

Program kembali melakukan koneksi ke database pets.db, kemudian mengeksekusi perintah SQL:

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet
for row in cur.execute("SELECT * FROM Pet"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()

(1, 'Felix', 'Alex')
(2, 'Auburn', 'Britt')
```

A text box in the bottom right corner contains the identifier: 25083010053_Nadia Siti Ramadhani

Perintah ini digunakan untuk mengambil semua data dari tabel. Hasil query kemudian ditampilkan pada terminal menggunakan perintah `print(row)`.

Setelah program dijalankan, terminal menampilkan seluruh isi tabel Pet, yaitu:

- (1, 'Felix', 'Alex')
- (2, 'Auburn', 'Britt')

Output ini menunjukkan bahwa proses pembuatan tabel, penambahan data, dan pengambilan data dari database SQLite menggunakan Python telah berjalan dengan baik.

➤ Mengurutkan Values Data pada Database

1. Mengurutkan values data dengan urutan descending/menurun menggunakan query DESC

Pada tahap ini dilakukan pengurutan data berdasarkan kolom PetName secara menurun (*descending*) menggunakan query SQL ORDER BY DESC.

Program Python kembali melakukan koneksi ke database pets.db, kemudian mengeksekusi perintah:

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet (pakai urutan descending)
for row in cur.execute("SELECT * FROM Pet ORDER BY PetName DESC"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

```
(1, 'Felix', 'Alex')
(2, 'Auburn', 'Britt')
```

25083010053_Nadia Siti Ramadhani

Query tersebut berfungsi untuk menampilkan seluruh data pada tabel **Pet** dengan urutan nama hewan dari huruf terakhir ke huruf pertama (Z ke A). Hasil query kemudian ditampilkan pada terminal menggunakan perintah `print(row)`. Hasil yang ditampilkan pada terminal menunjukkan bahwa data berhasil diurutkan secara menurun, yaitu:

- (1, 'Felix', 'Alex')
- (2, 'Auburn', 'Britt')

2. Sedangkan apabila mengurutkan data dengan urutan ascending tidak perlu menuliskannya pada query

Selanjutnya dilakukan pengurutan data secara menaik (*ascending*). Pada SQL, pengurutan ascending merupakan urutan default sehingga tidak perlu menuliskan kata **ASC** pada query. Program menjalankan perintah:

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet (pakai urutan ascending)
for row in cur.execute("SELECT * FROM Pet ORDER BY PetName"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

```
(2, 'Auburn', 'Britt')
(1, 'Felix', 'Alex')
```

25083010053_Nadia Siti Ramadhani

Query ini menampilkan data dari huruf A ke Z berdasarkan kolom **PetName**. Hasil yang ditampilkan pada terminal menunjukkan bahwa data telah diurutkan dari A ke Z, yaitu:

- (2, 'Auburn', 'Britt')
- (1, 'Felix', 'Alex')

➤ Memodifikasi Kolom pada Tabel

1. Mengubah Struktur Tabel Menggunakan Python

Tahap selanjutnya adalah melakukan modifikasi struktur tabel dengan menambahkan kolom baru pada tabel Pet menggunakan perintah SQL ALTER TABLE.

Program menjalankan dua perintah berikut:

- Menambahkan kolom Age bertipe integer
- Menambahkan kolom Gender bertipe varchar(1)

Perintah ini dijalankan melalui Python menggunakan `cursor.execute()`, kemudian perubahan disimpan dengan `commit()`.

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# digunakan untuk menambahkan kolom di tabel pet
for row in cur.execute("ALTER TABLE Pet ADD COLUMN Age int"):
    print(row)
for row in cur.execute("ALTER TABLE Pet ADD COLUMN Gender varchar(1)"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

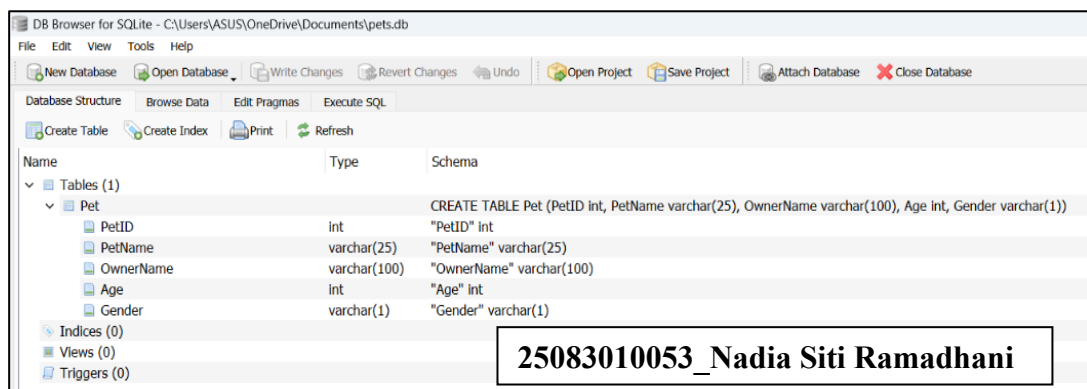
25083010053_Nadia Siti Ramadhani

2. Hasilnya dapat dilihat pada DB Browser for SQLite. Terdapat kolom baru yang telah ditambahkan melalui python

Setelah proses berhasil dijalankan, database dibuka kembali menggunakan **DB Browser for SQLite**. Pada bagian **Database Structure** terlihat bahwa tabel **Pet** kini memiliki dua kolom tambahan, yaitu:

- Age (int)
- Gender (varchar(1))

Hal ini menunjukkan bahwa proses modifikasi struktur tabel berhasil dilakukan.



25083010053_Nadia Siti Ramadhani

➤ Update Data

Langkah 1: Mengubah Nilai Data Menggunakan Query UPDATE

Setelah kolom Age dan Gender ditambahkan, kolom tersebut masih kosong sehingga perlu dilakukan proses pembaruan data (*update*). Proses update dilakukan

menggunakan perintah SQL UPDATE melalui Python. Program melakukan koneksi ke database pets.db, kemudian menjalankan dua perintah update berikut:

- Mengubah data pet bernama Auburn → Age = 2, Gender = F
- Mengubah data pet bernama Felix → Age = 3, Gender = M

Query yang digunakan:

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# digunakan untuk menambah value atau mengganti value yang sudah ada sebelumnya
for row in cur.execute("UPDATE Pet SET age = 2, gender = 'F' WHERE PetName = 'Auburn')":
    print(row)
for row in cur.execute("UPDATE Pet SET age = 3, gender = 'M' WHERE PetName = 'Felix')":
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

Perubahan disimpan menggunakan commit() agar tersimpan permanen di database.

Langkah 2: Mengecek Hasil Update Menggunakan SELECT

Untuk memastikan data telah diperbarui, dilakukan pengecekan menggunakan query:

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet
for row in cur.execute("SELECT * FROM Pet"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

```
(1, 'Felix', 'Alex', 3, 'M')
(2, 'Auburn', 'Britt', 2, 'F')
```

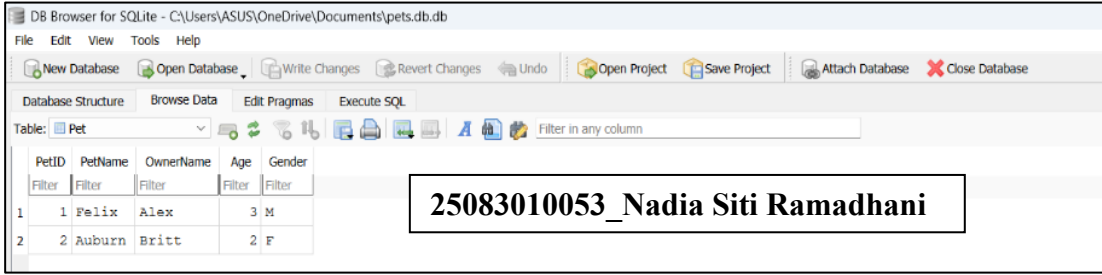
Hasil output pada terminal menunjukkan bahwa kolom **Age** dan **Gender** telah terisi sesuai dengan perintah update.

Data yang ditampilkan:

- (1, 'Felix', 'Alex', 3, 'M')
- (2, 'Auburn', 'Britt', 2, 'F')

Langkah 3: Verifikasi Hasil Update pada DB Browser for SQLite

Setelah hasil update ditampilkan pada terminal Python, tahap terakhir adalah melakukan pengecekan langsung pada database menggunakan DB Browser for SQLite. Database pets.db dibuka kembali, kemudian pada menu Browse Data dipilih tabel Pet. Hasil yang ditampilkan menunjukkan bahwa data pada kolom Age dan Gender telah terisi sesuai dengan hasil output di VSCode.



PetID	PetName	OwnerName	Age	Gender
1	Felix	Alex	3	M
2	Auburn	Britt	2	F

Isi tabel yang terlihat pada SQLite adalah:

- (1, Felix, Alex, 3, M)
- (2, Auburn, Britt, 2, F)

Hasil ini sama persis dengan output yang muncul pada terminal Python, sehingga dapat disimpulkan bahwa proses update data telah berhasil tersimpan secara permanen di dalam database.

➤ Menggunakan Group By untuk mengelompokkan data

Langkah 1: Menambahkan Data ke dalam Database SQLite

Pada tahap awal, dilakukan proses penambahan data ke dalam database SQLite menggunakan bahasa pemrograman Python dengan bantuan library sqlite3. Koneksi ke database pets.db dibuat terlebih dahulu, kemudian dilakukan eksekusi perintah SQL untuk menambahkan data baru ke dalam tabel *Pet*.

Perintah INSERT INTO digunakan untuk memasukkan data hewan beserta informasi PetID, PetName, OwnerName, Age, dan Gender. Data yang ditambahkan adalah sebagai berikut:

- (3, "Coco", "Bill", 2, "M")
- (4, "Kin", "Bob", 1, "F")

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk mengisi atau memasukkan nilai ke dalam tabel pet
for row in cur.execute("INSERT INTO Pet VALUES (3, 'Coco', 'Bill', 2, 'M')"):
    print(row)
for row in cur.execute("INSERT INTO Pet VALUES (4, 'Kin', 'Bob', 1, 'F')"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

Perintah dijalankan melalui cursor, kemudian perubahan disimpan menggunakan fungsi `commit()` agar data tersimpan secara permanen di dalam database. Setelah itu, koneksi ke database ditutup. Tahap ini bertujuan untuk menambahkan data baru yang akan digunakan dalam proses pengolahan data selanjutnya.

Langkah 2: Menampilkan Data dari Tabel

Setelah data berhasil ditambahkan, langkah selanjutnya adalah menampilkan seluruh isi tabel untuk memastikan bahwa data telah tersimpan dengan benar. Query yang digunakan adalah:

```
SELECT * FROM Pet ORDER BY Age DESC
```

Perintah ini menampilkan seluruh data pada tabel *Pet* dengan pengurutan berdasarkan umur (*Age*) dari yang terbesar ke terkecil.

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet (pakai urutan descending)
for row in cur.execute("SELECT*FROM Pet ORDER BY age DESC"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

```
(1, 'Felix', 'Alex', 3, 'M')
(2, 'Auburn', 'Britt', 2, 'F')
(3, 'Coco', 'Bill', 2, 'M')
(4, 'Kin', 'Bob', 1, 'F')
```

Hasil yang ditampilkan pada terminal Python adalah:

- (1, 'Felix', 'Alex', 3, 'M')
- (2, 'Auburn', 'Britt', 2, 'F')
- (3, 'Coco', 'Bill', 2, 'M')
- (4, 'Kin', 'Bob', 1, 'F')

Hasil tersebut menunjukkan bahwa data telah tersimpan dengan lengkap dan urutan data sesuai dengan kriteria yang ditentukan.



The screenshot shows the DB Browser for SQLite interface. The 'Table: Pet' is selected, and the data is displayed in a table. The table has columns: PetID, PetName, OwnerName, Age, and Gender. The data is sorted by Age in descending order.

PetID	PetName	OwnerName	Age	Gender
1	Felix	Alex	3	M
2	Auburn	Britt	2	F
3	Coco	Bill	2	M
4	Kin	Bob	1	F

Setelah data berhasil ditambahkan, langkah selanjutnya adalah menampilkan seluruh isi tabel untuk memastikan bahwa data telah tersimpan dengan benar. Pada tahap ini, data ditampilkan menggunakan aplikasi **DB Browser for SQLite** pada menu *Browse Data*. Perintah ini menampilkan seluruh data pada tabel Pet dengan pengurutan berdasarkan umur (*Age*) dari yang terbesar ke terkecil.

Langkah 3: Mengelompokkan Data Menggunakan GROUP BY

Pada tahap ini dilakukan proses pengelompokan data berdasarkan jenis kelamin (*Gender*) menggunakan klausa GROUP BY. Selain itu, digunakan fungsi agregasi COUNT() untuk menghitung jumlah data pada masing-masing kelompok. Query yang digunakan adalah SELECT COUNT(*), Gender FROM Pet GROUP BY Gender.

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet (pakai urutan descending)
for row in cur.execute("SELECT COUNT(*), gender FROM Pet GROUP BY gender"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

```
(2, 'F')
(2, 'M')
```

Hasil yang diperoleh adalah:

- (2, M)
- (2, F)

Hasil ini menunjukkan bahwa terdapat 2 data dengan jenis kelamin jantan (M) dan 2 data dengan jenis kelamin betina (F). Dengan demikian, proses pengelompokan data dan perhitungan jumlah data pada masing-masing kategori telah berhasil dilakukan.

➤ Relation Mapping in Database

Langkah 1: Membuat Relasi Antar Tabel

Pada tahap ini dilakukan pembuatan tabel baru yang memiliki relasi dengan tabel Pet. Relasi dibuat menggunakan foreign key yang menghubungkan kolom PetID pada tabel baru dengan PetID pada tabel Pet. Tujuan dari relasi ini adalah untuk menjaga keterkaitan data antar tabel, sehingga setiap data pada tabel baru pasti merujuk pada data yang valid di tabel Pet. Selain itu, digunakan aturan ON DELETE CASCADE agar data yang berhubungan ikut terhapus secara otomatis ketika data induknya dihapus.

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# digunakan untuk membuat tabel PetTreatment
cur.execute("PRAGMA foreign_keys = 1")
sql = """CREATE TABLE PetTreatment (PetID int, TreatDate date, TreatResult varchar(255), CONSTRAINT fk_pet FOREIGN KEY (PetID) REFERENCES Pet(PetID))"""
cur.execute(sql)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

Langkah 2: Verifikasi pada DB Browser

Setelah tabel berhasil dibuat, dilakukan pengecekan menggunakan DB Browser for SQLite. Pada tampilan database terlihat bahwa tabel baru telah berhasil ditambahkan dan berdampingan dengan tabel Pet.

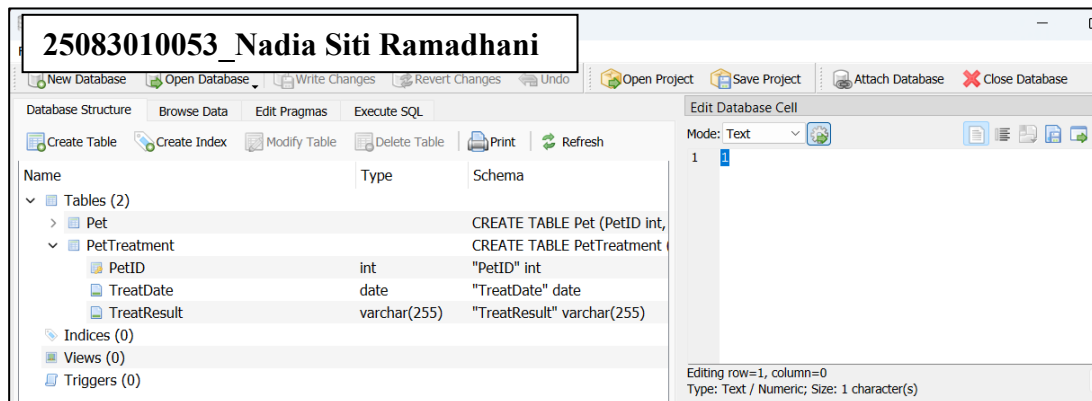
Pada tampilan bagian *Database Structure*, terlihat bahwa sekarang terdapat dua tabel, yaitu:

- **Pet**
- **PetTreatment**

Tabel PetTreatment menampilkan struktur kolom sebagai berikut:

- **PetID (int)**

- **TreatDate** (date)
- **TreatResult** (varchar(255))



Hal ini menunjukkan bahwa proses pembuatan relasi telah berhasil, serta struktur database sudah terbentuk dengan baik dan saling terhubung antar tabel.

➤ Menampilkan kolom yang spesifik dengan query join

Langkah 1: Menambahkan Data Pendukung

Pada tahap ini dilakukan penambahan data ke dalam tabel PetTreatment. Data yang dimasukkan berupa informasi tanggal perawatan (TreatDate) dan hasil perawatan (TreatResult) untuk setiap hewan.

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

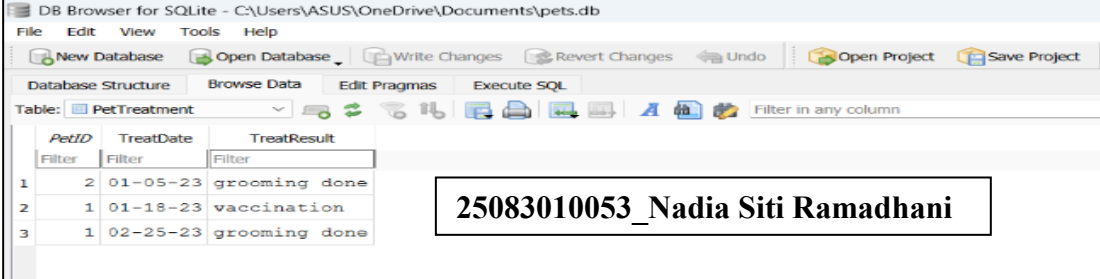
# untuk menambahkan isi atau value ke tabel pettreatment
cur.execute("INSERT INTO PetTreatment VALUES (2, '01-05-23', 'grooming done')")
cur.execute("INSERT INTO PetTreatment VALUES (1, '01-18-23', 'vaccination')")
cur.execute("INSERT INTO PetTreatment VALUES (1, '01-25-23', 'grooming done')")

con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

Penambahan data ini penting karena tabel PetTreatment akan digunakan bersama dengan tabel Pet. Setiap data pada tabel ini terhubung melalui PetID, sehingga nantinya dapat digabungkan untuk menampilkan informasi yang lebih lengkap.



DB Browser for SQLite - C:\Users\ASUS\OneDrive\Documents\pets.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project

Database Structure Browse Data Edit Pragmas Execute SQL

Table: PetTreatment

PetID	TreatDate	TreatResult
Filter	Filter	Filter
1	2 01-05-23	grooming done
2	1 01-18-23	vaccination
3	1 02-25-23	grooming done

25083010053_Nadia Siti Ramadhani

Selain itu, hasil dari proses *insert* tersebut dapat dilihat melalui menu *Browse Data* pada DB Browser for SQLite. Pada tampilan tersebut terlihat bahwa data perawatan telah berhasil ditambahkan ke dalam tabel PetTreatment, sesuai dengan perintah yang dijalankan pada Python. Hal ini menunjukkan bahwa proses penyimpanan data ke database telah berjalan dengan baik dan siap digunakan untuk proses *JOIN*.

Langkah 2: Menampilkan Data dengan JOIN

Setelah data tersedia, dilakukan penggabungan data dari tabel Pet dan PetTreatment menggunakan query *JOIN*. Penggabungan ini dilakukan berdasarkan kesamaan nilai PetID pada kedua tabel. Tujuan penggunaan *JOIN* adalah untuk menampilkan data yang saling berhubungan dalam satu tampilan. Dari hasil yang diperoleh, ditampilkan beberapa kolom seperti nama hewan (*PetName*), nama pemilik (*OwnerName*), dan hasil perawatan (*TreatResult*).

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet dan pettreatment yang berelasi
for row in cur.execute("SELECT p.PetName, p.OwnerName, t.TreatResult FROM Pet as p join PetTreatment as t ON p.PetID = t.PetID"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

```
('Felix', 'Alex', 'grooming done')
('Felix', 'Alex', 'vaccination')
('Auburn', 'Britt', 'grooming done')
```

Hasil tersebut menunjukkan bahwa data dari dua tabel berbeda dapat digabungkan dengan baik, sehingga informasi yang dihasilkan menjadi lebih lengkap dibandingkan jika hanya menggunakan satu tabel saja.

Langkah 3: Menampilkan Data dengan Format Lebih Informatif

Pada tahap ini, hasil query tidak hanya ditampilkan dalam bentuk data mentah, tetapi diolah menjadi format yang lebih mudah dipahami. Data ditampilkan dalam bentuk kalimat yang menjelaskan informasi secara lengkap. Dengan cara ini, pengguna dapat langsung memahami isi data tanpa perlu membaca struktur tabel. Informasi seperti nama hewan, pemilik, tanggal perawatan, dan hasil perawatan dapat disajikan secara lebih jelas dan komunikatif.

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

rows = cur.execute("SELECT p.PetName, p.OwnerName, t.TreatDate, t.TreatResult FROM Pet as p JOIN PetTreatment as t ON p.PetID = t.PetID").fetchall()
for row in rows:
    PetName = row[0]
    OwnerName = row[1]
    TreatDate = row[2]
    TreatResult = row[3]
    print("Nama " + PetName + " milik " + OwnerName + " pada " + TreatDate + " mendapatkan " + TreatResult)

con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

```
Nama Felix milik Alex pada 01-18-23 mendapatkan vaccination
Nama Felix milik Alex pada 01-25-23 mendapatkan grooming done
Nama Auburn milik Britt pada 01-05-23 mendapatkan grooming done
```

➤ Menghapus baris pada tabel

Sebelum dilakukan proses penghapusan data, terlebih dahulu ditampilkan seluruh isi tabel *Pet* untuk mengetahui kondisi awal data yang tersimpan di dalam database. Dari hasil tersebut terlihat bahwa terdapat data dengan **PetName "Coco"** yang masih tersimpan di dalam tabel. Data inilah yang akan dihapus pada tahap berikutnya.

25083010053_Nadia Siti Ramadhani

	PetID	PetName	OwnerName	Age	Gender
	Filter	Filter	Filter	Fil...	Filter
1	1	Felix	Alex	3	M
2	2	Auburn	Britt	2	F
3	3	Coco	Bill	2	M
4	4	Kin	Bob	1	F

Langkah 1: Menghapus Data dari Tabel

Setelah mengetahui kondisi awal data, langkah selanjutnya adalah melakukan penghapusan data menggunakan perintah DELETE FROM.

Query yang digunakan adalah:

```
with sqlite3.connect("pets.db", timeout=10) as con:
    cur = con.cursor()
    cur.execute("DELETE FROM pet WHERE PetName='Coco'")
    con.commit()
```

25083010053_Nadia Siti Ramadhani

Perintah tersebut berfungsi untuk menghapus baris data yang memiliki nilai *PetName* = "Coco". Setelah query dijalankan, perubahan disimpan menggunakan commit() agar penghapusan data bersifat permanen di dalam database.

Langkah 2: Menampilkan Data Setelah Penghapusan

Setelah proses penghapusan dilakukan, data ditampilkan kembali untuk memastikan bahwa data dengan *PetName* "Coco" telah berhasil dihapus. Query yang digunakan adalah:

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet
for row in cur.execute("SELECT * FROM Pet"):
    print(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

```
(1, 'Felix', 'Alex', 3, 'M')
(2, 'Auburn', 'Britt', 2, 'F')
(4, 'Kin', 'Bob', 1, 'F')
```

Terlihat bahwa data dengan nama "Coco" sudah tidak ada lagi dalam tabel. Selanjutnya dilakukan verifikasi menggunakan DB Browser for SQLite. Pada menu *Browse Data*, tabel *Pet* ditampilkan dan hasilnya menunjukkan data yang sama seperti output pada terminal Python, yaitu tanpa adanya data "Coco". Dengan demikian, dapat disimpulkan

bahwa proses penghapusan data menggunakan perintah DELETE FROM telah berhasil dilakukan dan perubahan telah tersimpan secara permanen di dalam database.



The screenshot shows the 'DB Browser for SQLite' application window. The 'Table: Pet' is selected, and the data is displayed in a table with columns: PetID, PetName, OwnerName, Age, and Gender. The data rows are: (1, Felix, Alex, 3, M), (2, Auburn, Britt, 2, F), and (3, Rin, Bob, 1, F). A text box with the identifier '25083010053_Nadia Siti Ramadhani' is overlaid on the right side of the table.

PetID	PetName	OwnerName	Age	Gender
1	Felix	Alex	3	M
2	Auburn	Britt	2	F
3	Rin	Bob	1	F

➤ RDBMS dan DATAFRAMES

Langkah 1: Import Library dan Inisialisasi Kolom DataFrame

Pada tahap awal, dilakukan persiapan untuk membuat dataframe dengan menggunakan library pandas. Library ini di-import terlebih dahulu agar dapat digunakan dalam proses pengolahan data.

```
import pandas as pd
column = ["PetID", "PetName", "OwnerName", "Age", "Gender", "TreatDate", "TreatResult"]
data = []
```

25083010053_Nadia Siti Ramadhani

Selanjutnya, ditentukan nama-nama kolom yang akan digunakan dalam *dataframe*, yaitu *PetID*, *PetName*, *OwnerName*, *Age*, *Gender*, *TreatDate*, dan *TreatResult*. Selain itu, dibuat sebuah list kosong bernama *data* yang berfungsi sebagai tempat penyimpanan sementara hasil pengambilan data dari database. Tahap ini bertujuan untuk menyiapkan struktur data sebelum dilakukan proses pengambilan data.

Langkah 2: Mengambil Data dari Database

Setelah struktur data disiapkan, langkah berikutnya adalah mengambil data dari database SQLite menggunakan query SQL. Query yang digunakan bertujuan untuk menampilkan data dari tabel *Pet* beserta informasi tambahan terkait perawatan. Proses pengambilan data dilakukan menggunakan cursor, di mana setiap baris hasil eksekusi query dimasukkan ke dalam list *data* menggunakan fungsi *append()*. Setelah seluruh data berhasil diambil, koneksi ke database ditutup. Tahap ini bertujuan untuk memindahkan data dari database ke dalam lingkungan Python sehingga dapat diolah lebih lanjut.

```
import sqlite3

# membuat koneksi SQL ke database SQLite
con = sqlite3.connect("pets.db", timeout=10)

cur = con.cursor()

# untuk menampilkan isi dari tabel pet dan pettreatment yang berelasi
rows = cur.execute("SELECT p.PetID, p.PetName, p.OwnerName, p.Age, p.Gender, t.TreatDate, t.TreatResult FROM Pet as p JOIN PetTreatment as t ON p.PetID = t.PetID")
for row in rows:
    data.append(row)
con.commit()

# pastikan untuk menutup koneksi
con.close()
```

25083010053_Nadia Siti Ramadhani

Langkah 3: Membuat DataFrame dari Data

Pada tahap ini, data yang telah dikumpulkan dalam list kemudian dikonversi menjadi sebuah *dataframe* menggunakan fungsi *DataFrame* dari library pandas. Proses ini dilakukan dengan memasukkan list *data* sebagai isi data serta variabel *column* sebagai nama kolomnya. Hasil yang diperoleh berupa sebuah *dataframe* yang menampilkan data hewan beserta informasi pemilik, umur, jenis kelamin, serta riwayat perawatan seperti tanggal dan hasil perawatan.

```
df = pd.DataFrame(data, columns=column)
df
```

25083010053_Nadia Siti Ramadhani

	PetID	PetName	OwnerName	Age	Gender	TreatDate	TreatResult
0	1	Felix	Alex	3	M	01-18-23	vaccination
1	1	Felix	Alex	3	M	01-25-23	grooming done
2	2	Auburn	Britt	2	F	01-05-23	grooming done

Contoh data yang ditampilkan menunjukkan bahwa satu hewan dapat memiliki lebih dari satu riwayat perawatan. Dengan demikian, dapat disimpulkan bahwa data dari database telah berhasil diubah ke dalam bentuk *dataframe* dan siap digunakan untuk proses analisis data lebih lanjut.